

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Heterogeneous Architectures and Accelerators (Part 2)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan

Last Lecture

- GPU coherence
- CUDA programming
- Revisiting Amdahl's Law
- Modeling accelerators
- Neural network accelerators

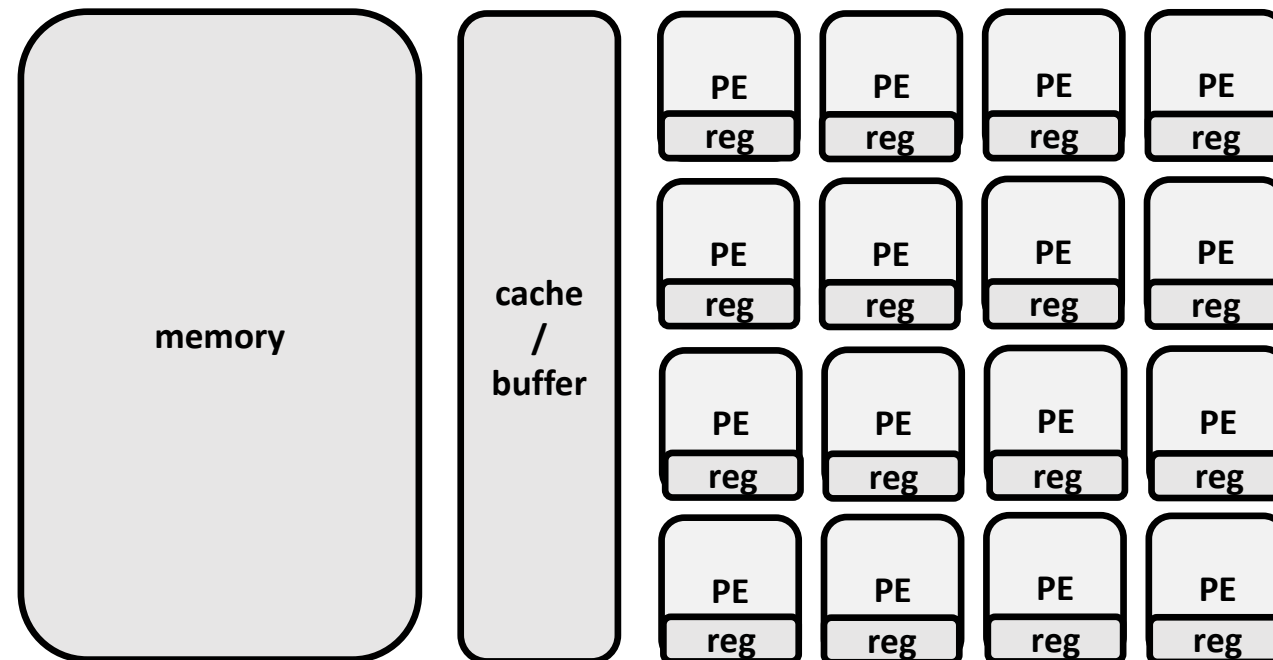
This Lecture

- Neural network accelerators
- Compute caches
- Computational skimming
- Unary computing

Neural Network Accelerators – Architecture

General architecture of NN acceleration:

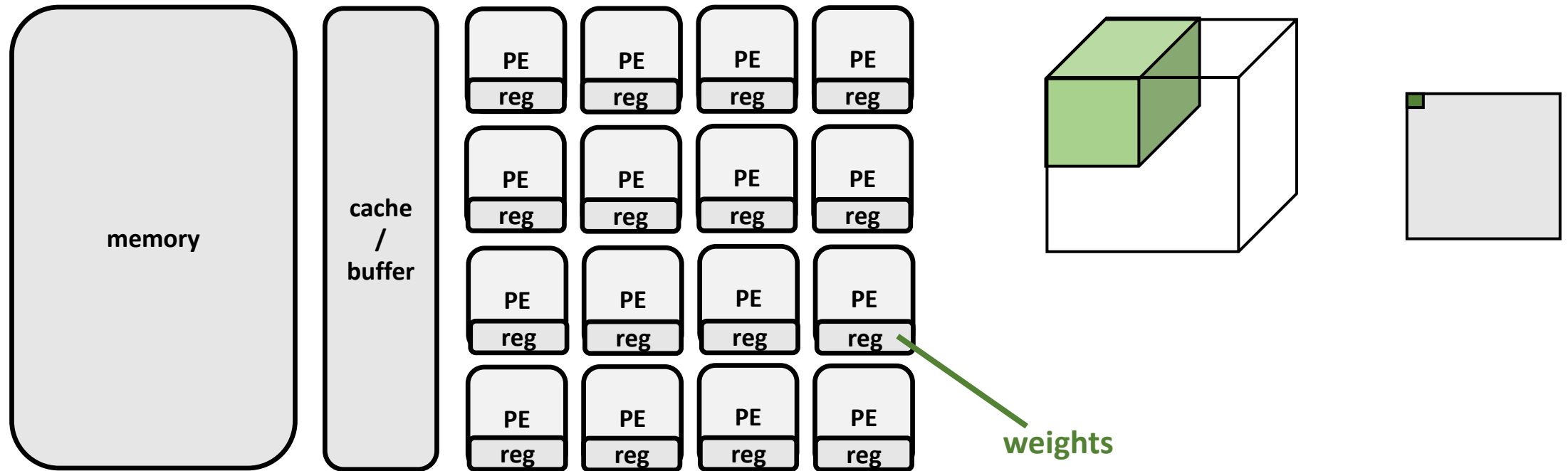
- Processing elements (PEs) with support for multiply-accumulate (MAC) and registers
- Software- or hardware-managed cache or scratchpad buffer
- Programmable network-on-chip to support *dataflow* of current NN layer



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

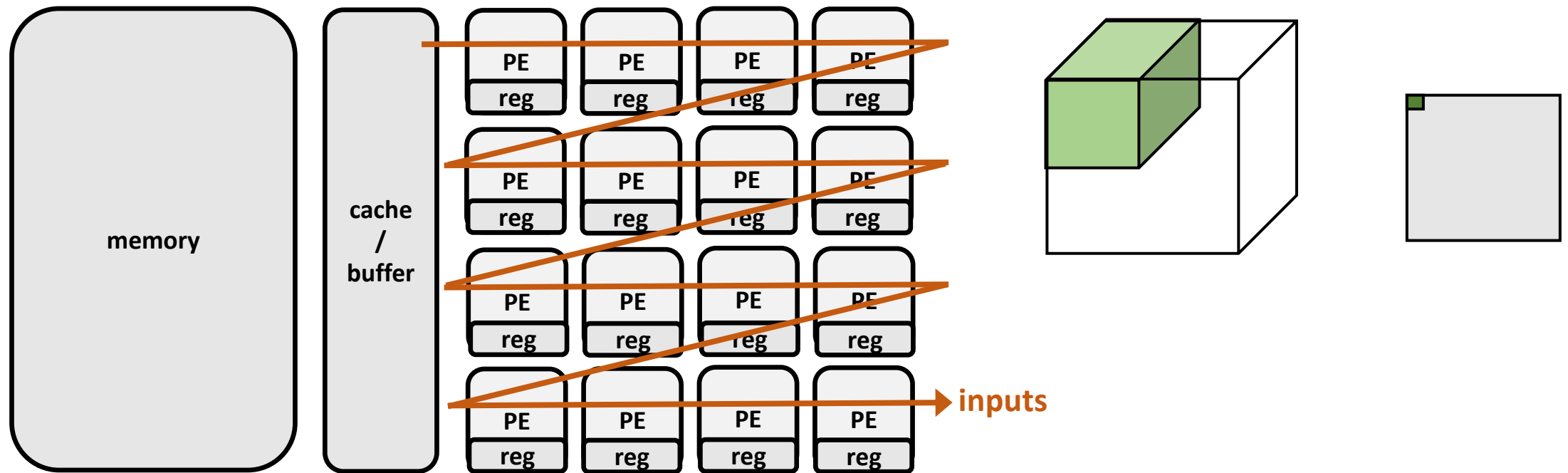
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

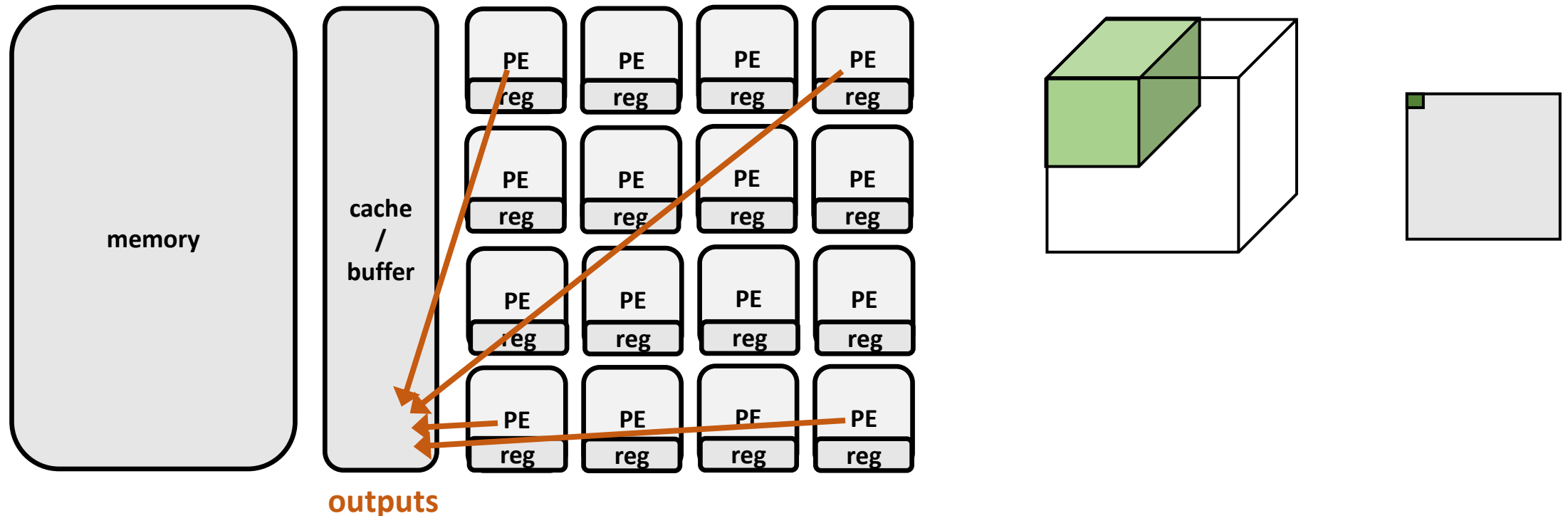
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

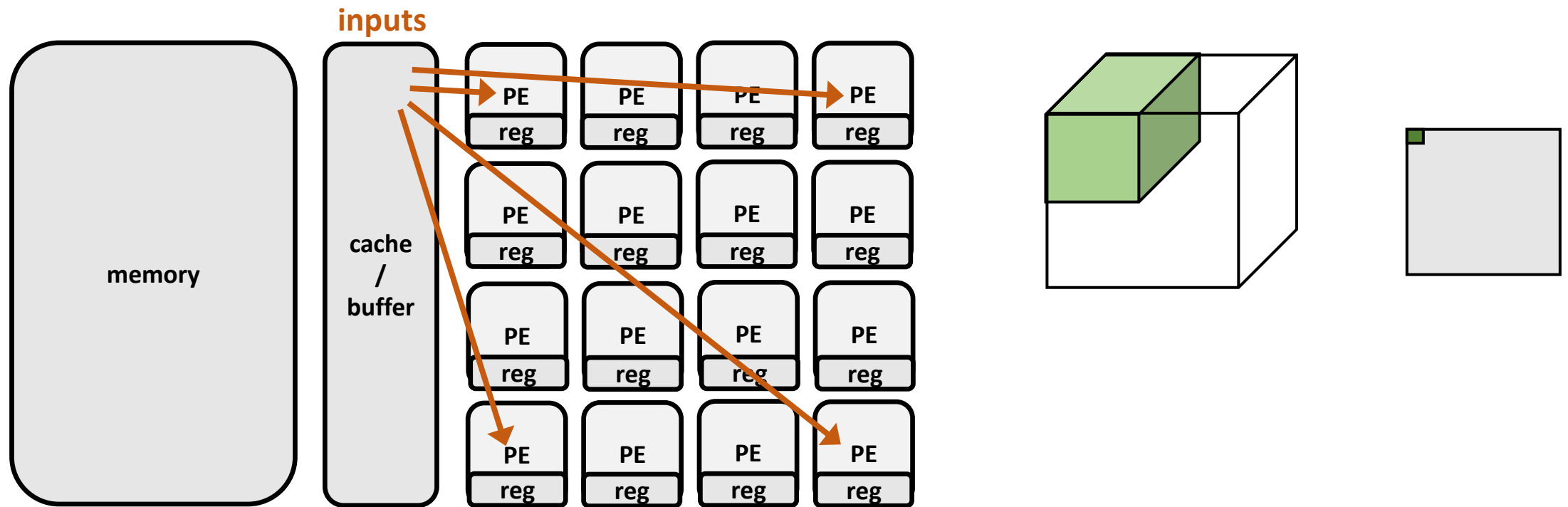
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

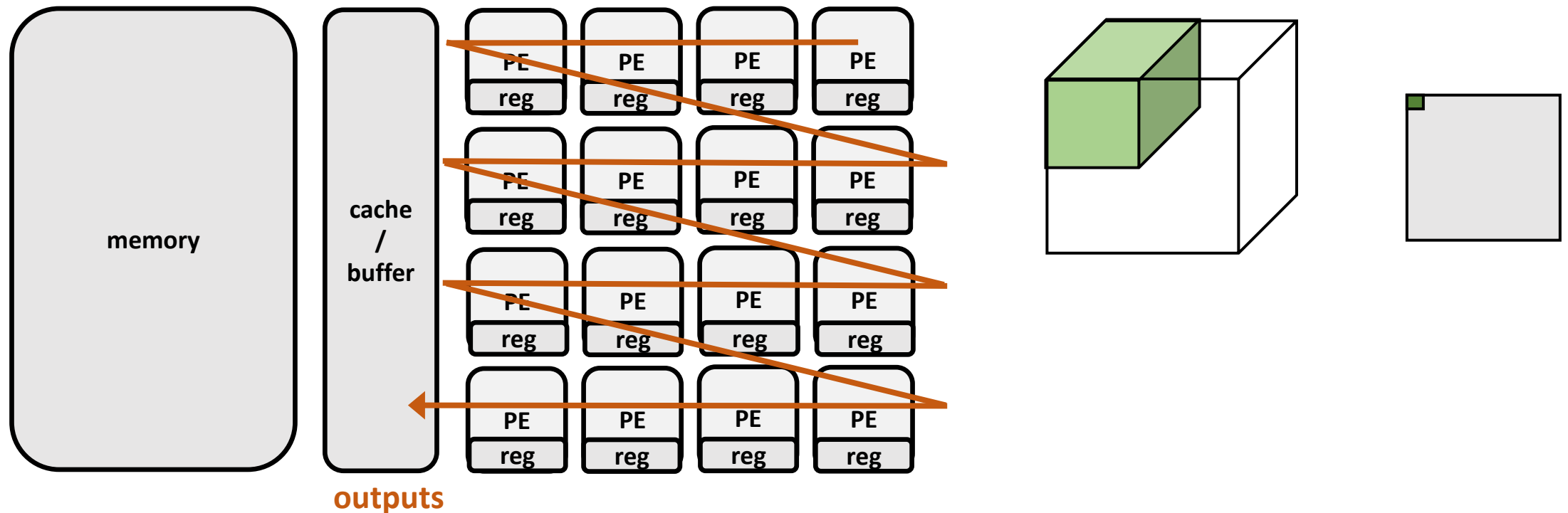
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

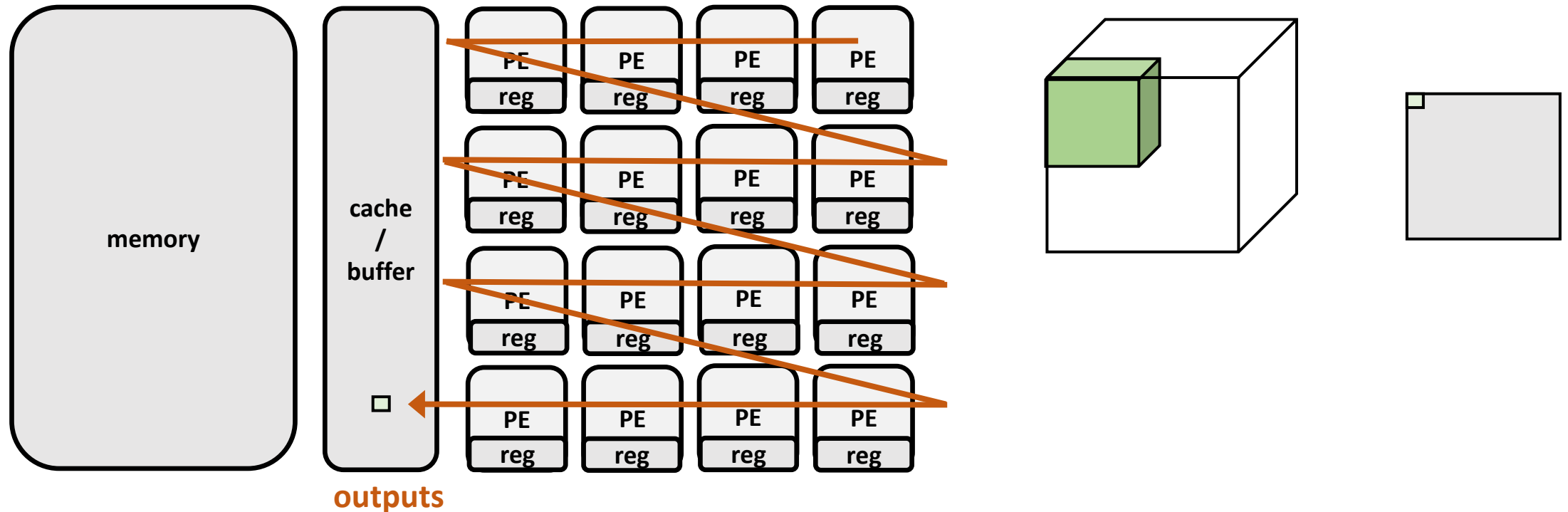
- PEs compute partial products for processing a single output element



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

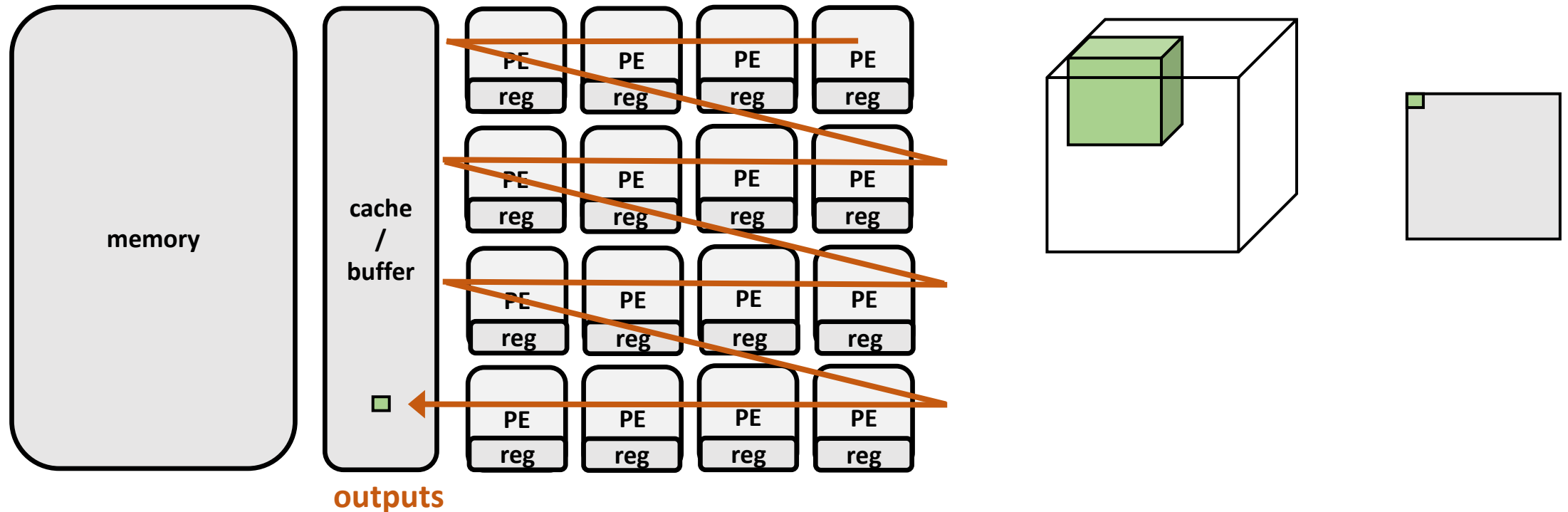
- PEs compute partial products for processing a single output element
- PE array should be large enough to hold filter, else lots of partial sums to buffer



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

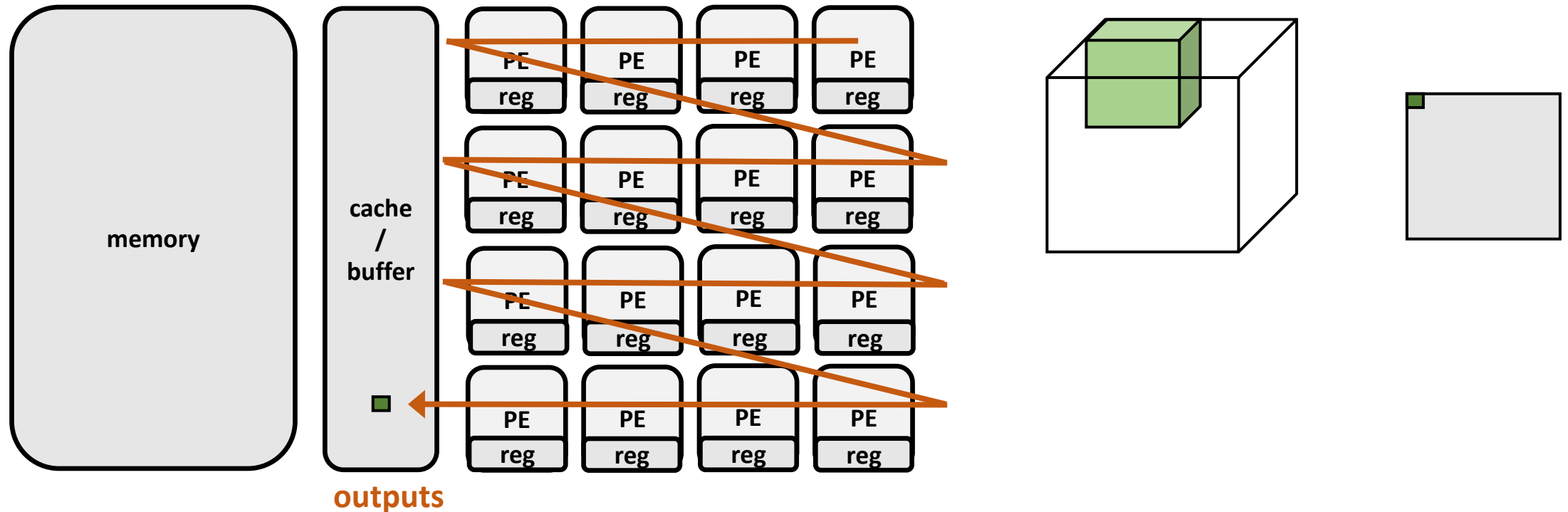
- PEs compute partial products for processing a single output element
- PE array should be large enough to hold filter, else lots of partial sums to buffer



Neural Network Accelerators – Architecture

Weight-Stationary Dataflow: Optimize for convolutional/batch weight reuse

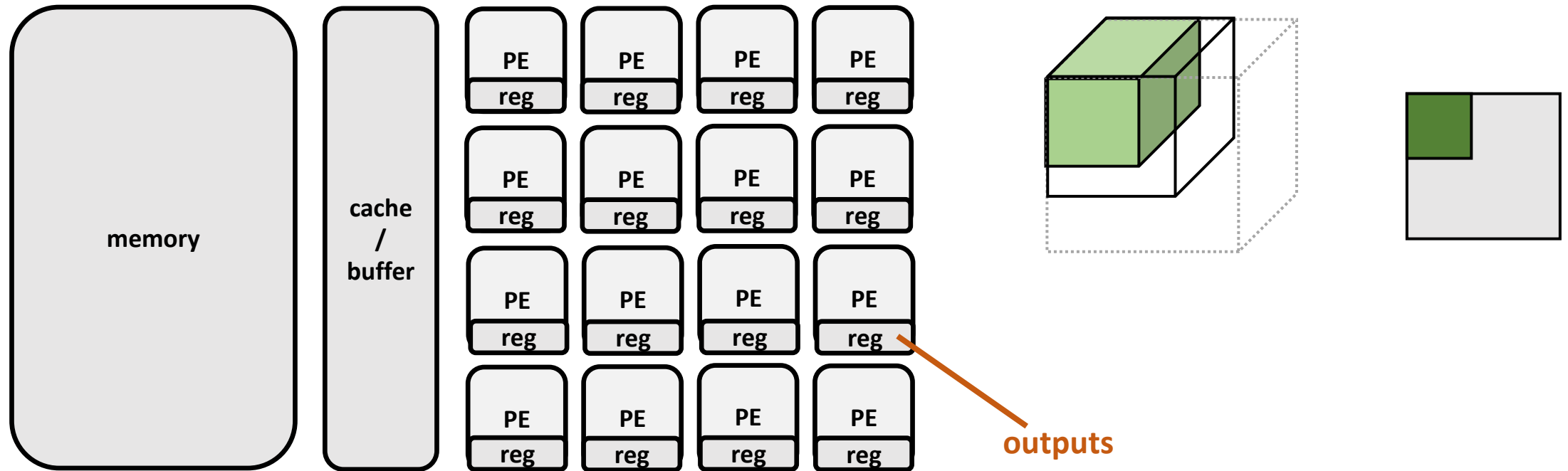
- PEs compute partial products for processing a single output element
- PE array should be large enough to hold filter, else lots of partial sums to buffer



Neural Network Accelerators – Architecture

Output-Stationary Dataflow: Optimize for output reuse

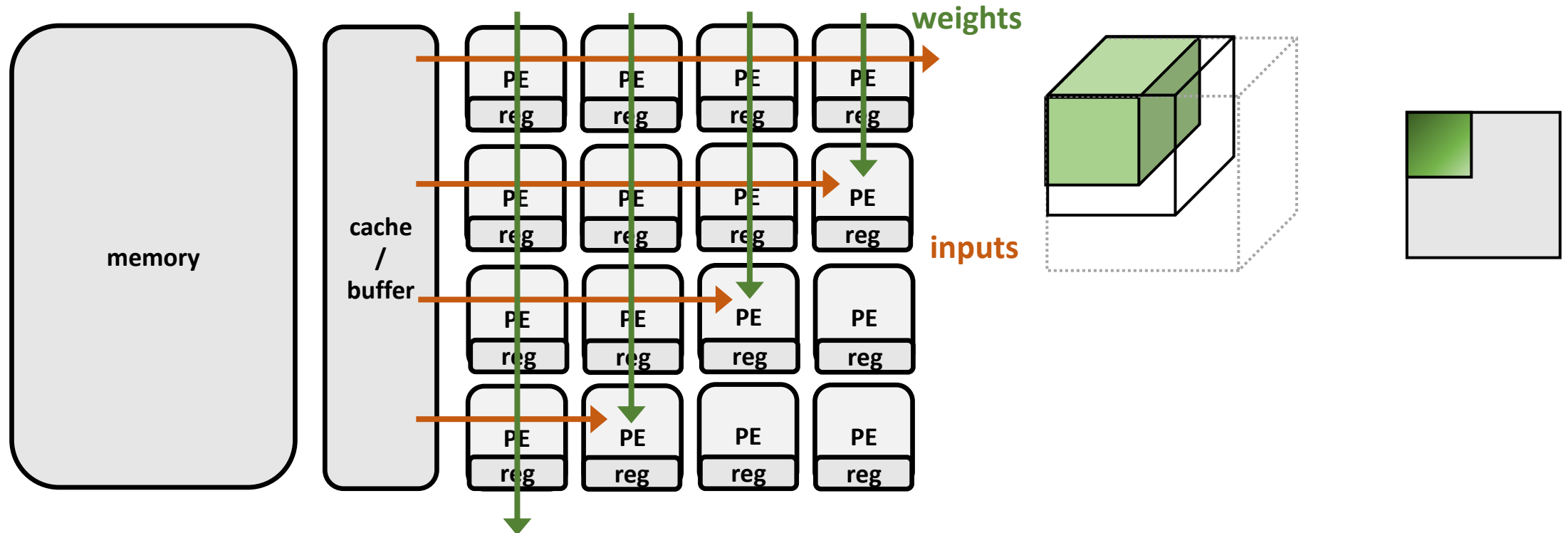
- PEs accumulate full products for a window of output elements (e.g., systolic array)



Neural Network Accelerators – Architecture

Output-Stationary Dataflow: Optimize for output reuse

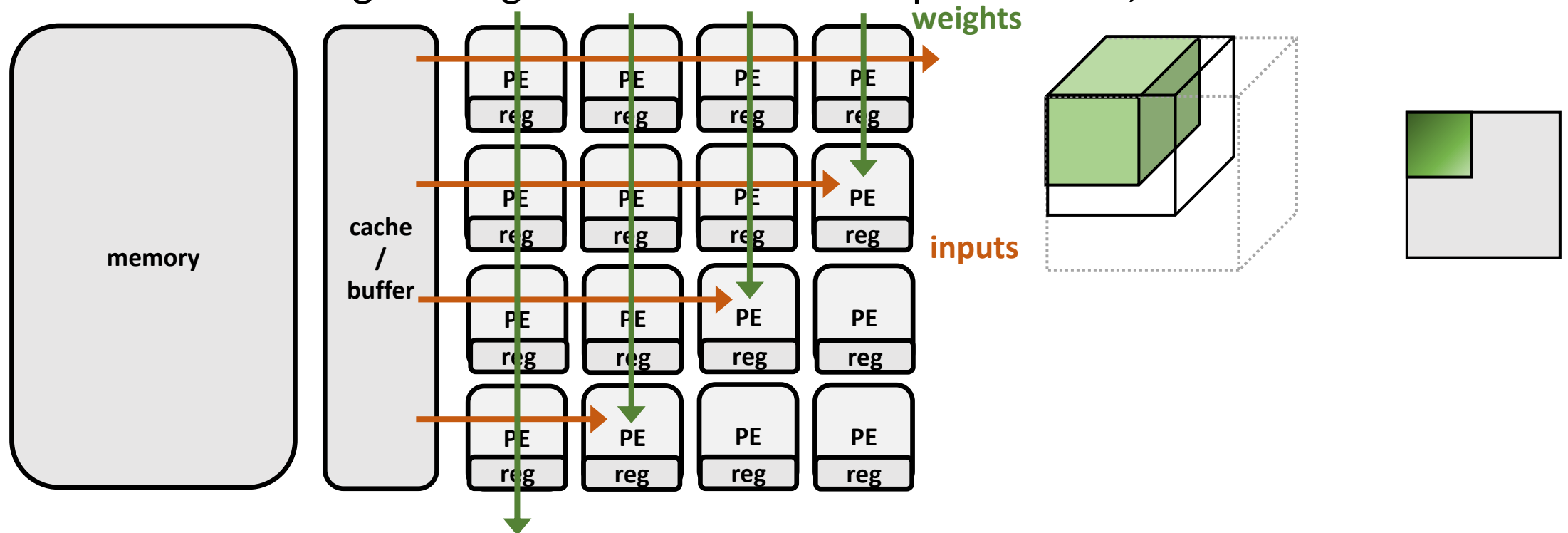
- PEs accumulate full products for a window of output elements (e.g., systolic array)



Neural Network Accelerators – Architecture

Output-Stationary Dataflow: Optimize for output reuse

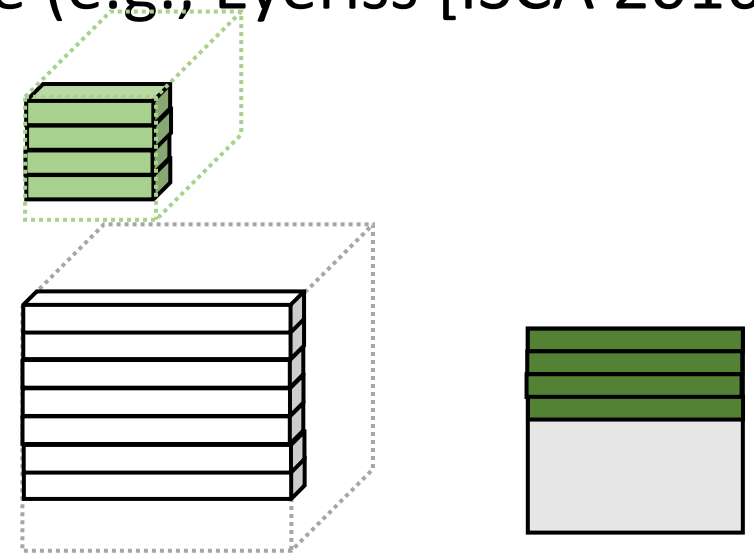
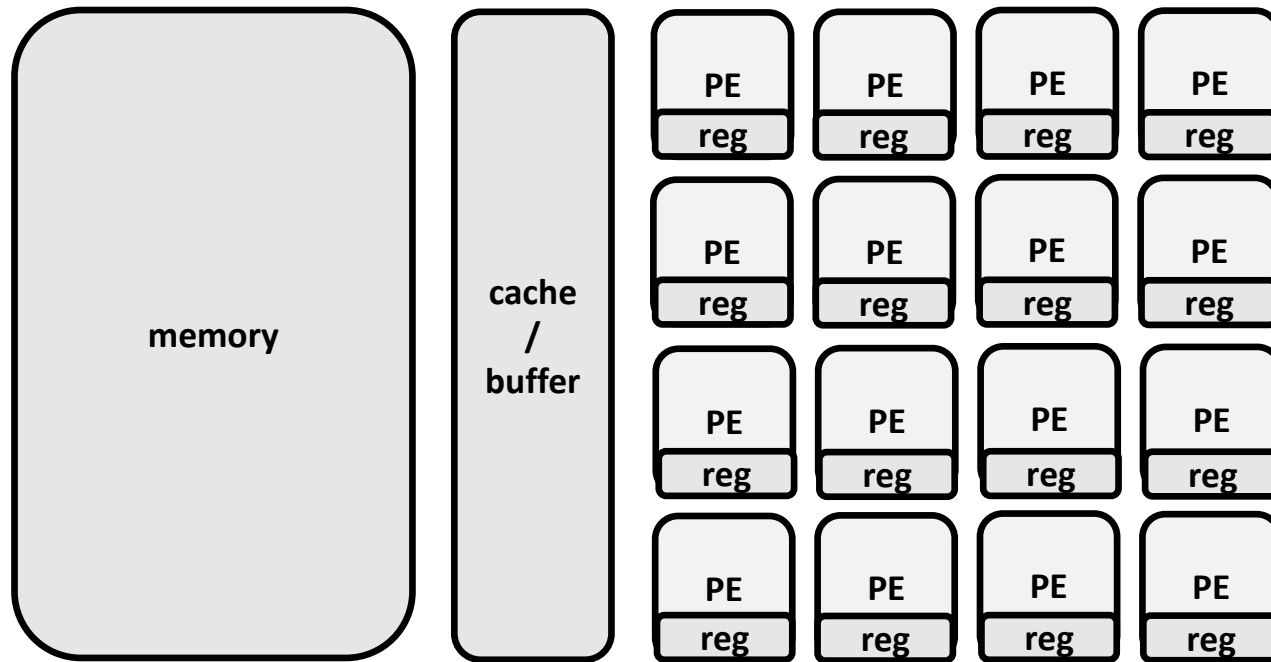
- PEs accumulate full products for a window of output elements (e.g., systolic array)
- Buffer should be large enough to hold filter and input window, else lots of mem traffic



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

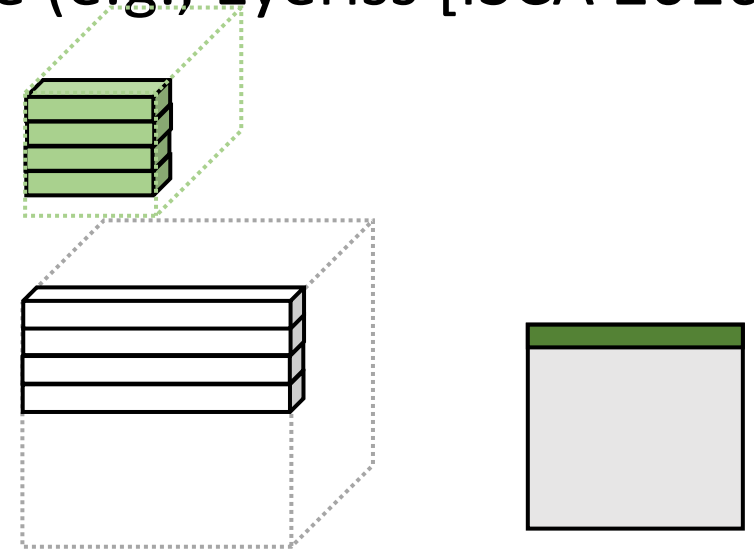
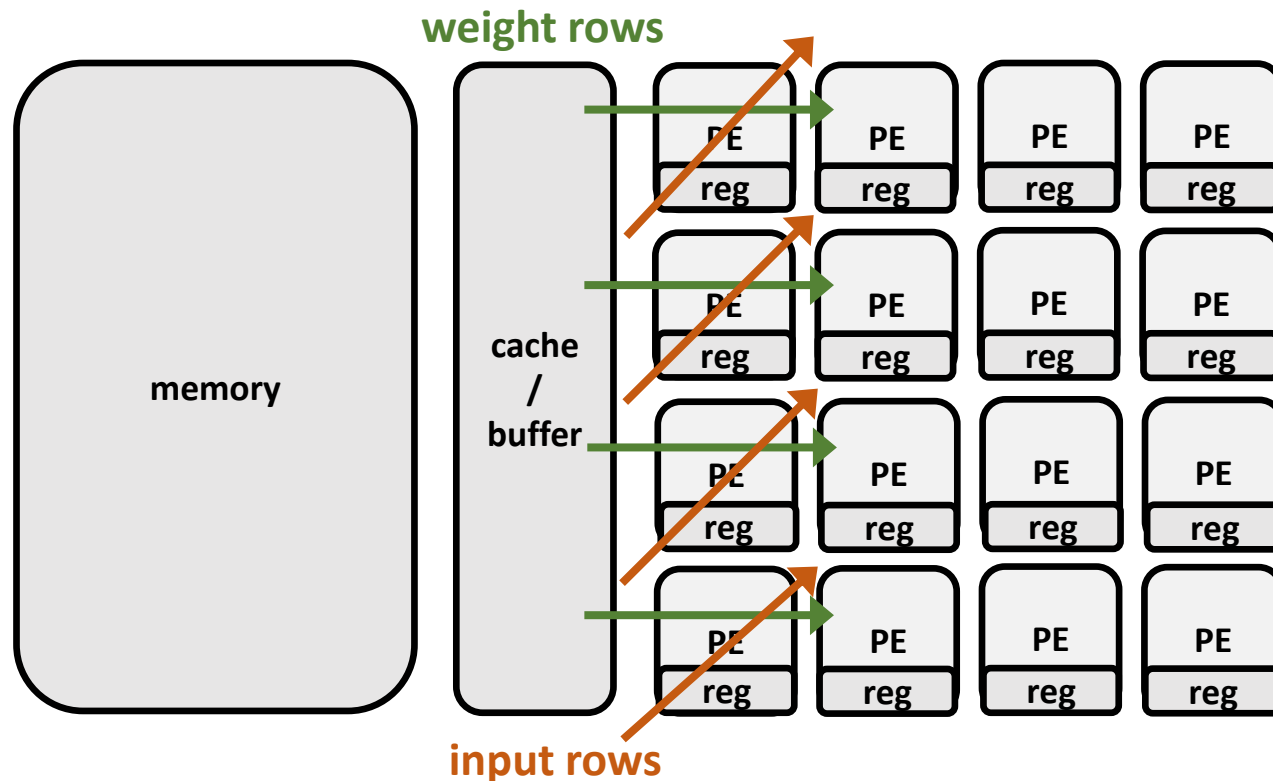
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

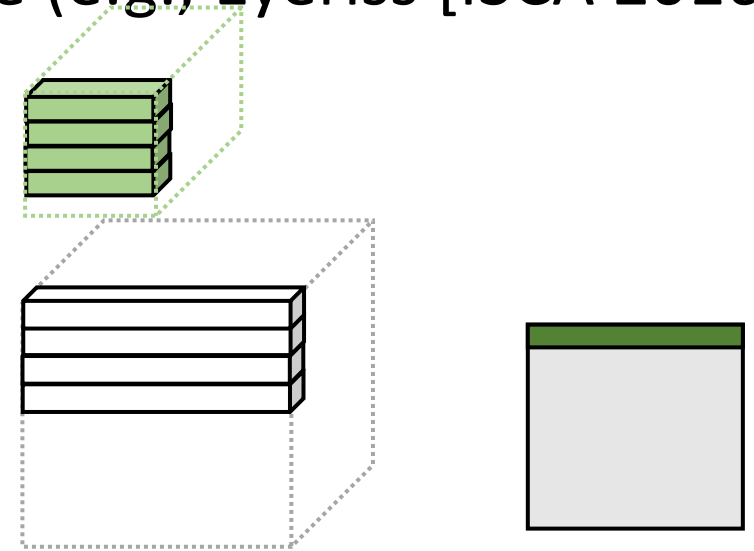
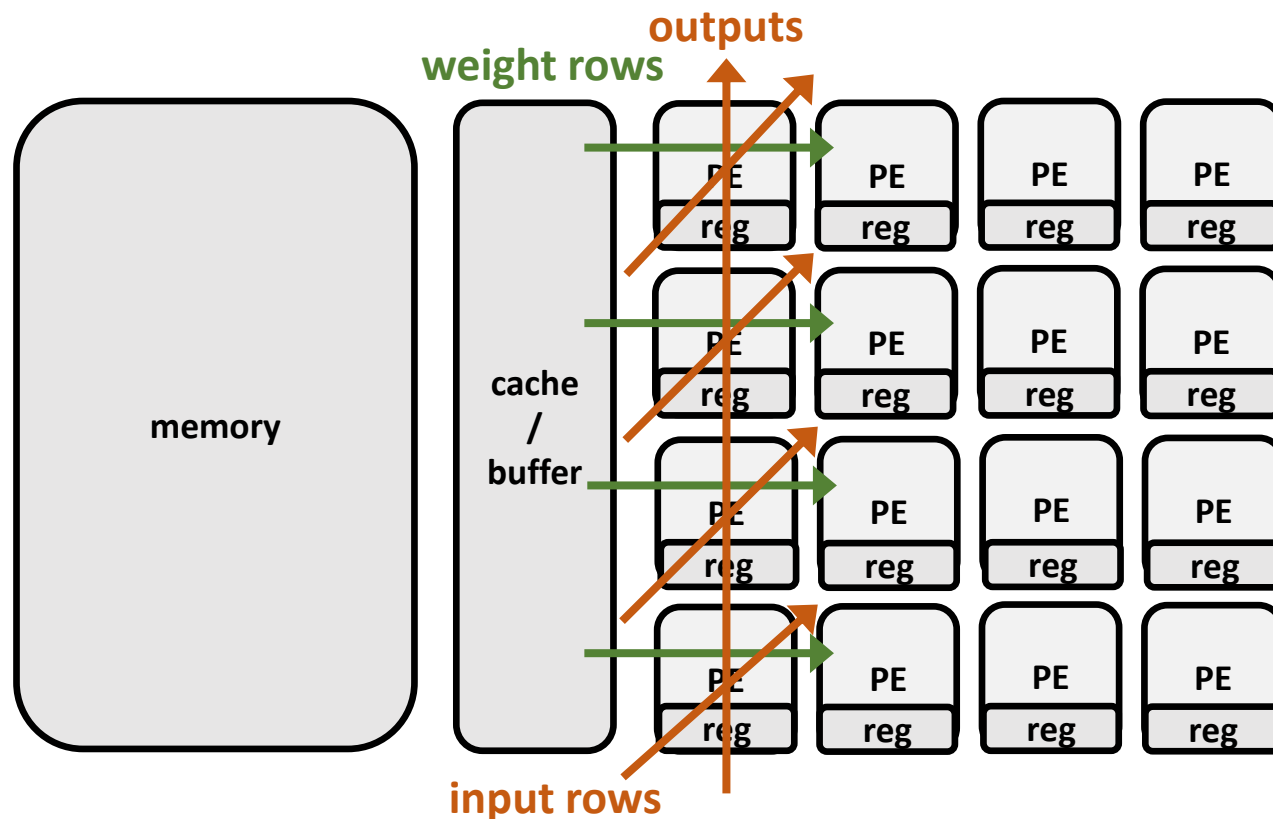
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

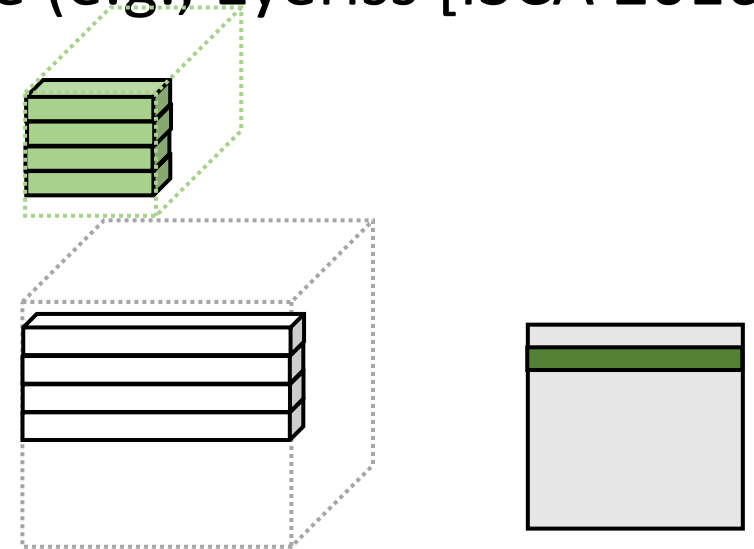
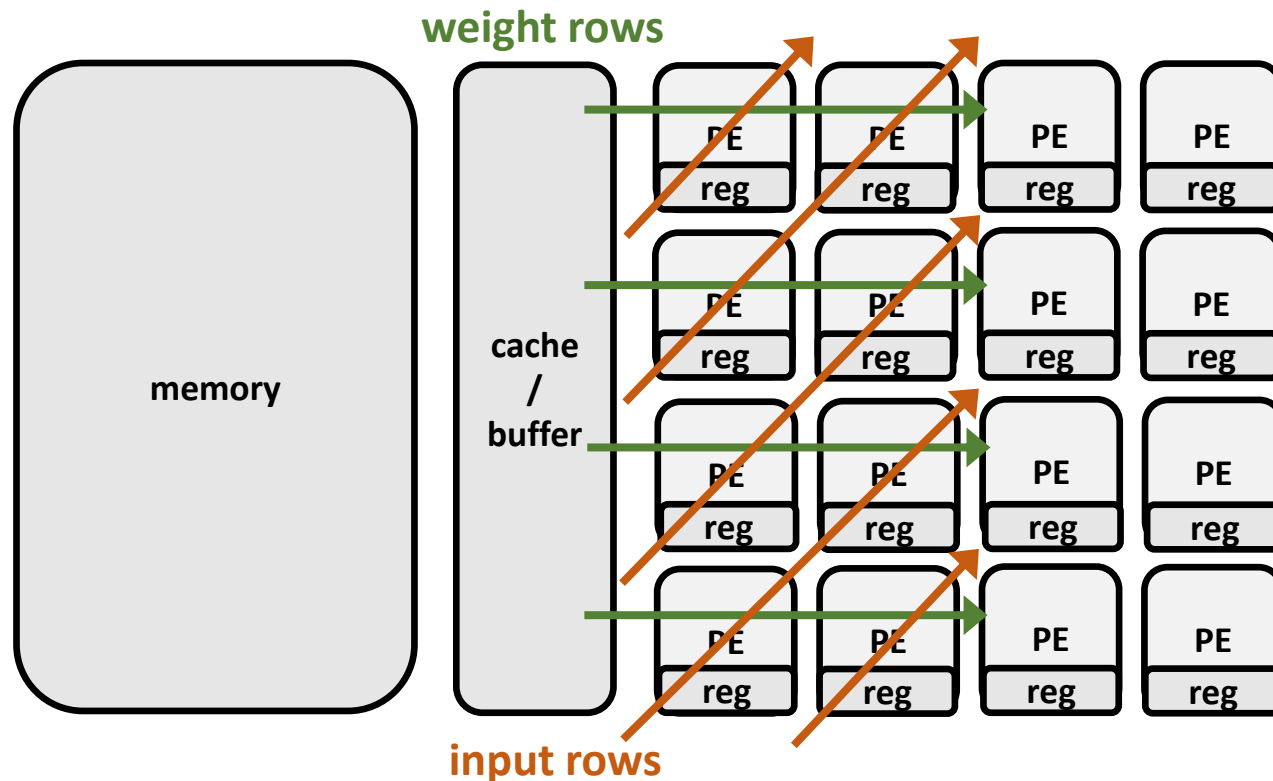
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

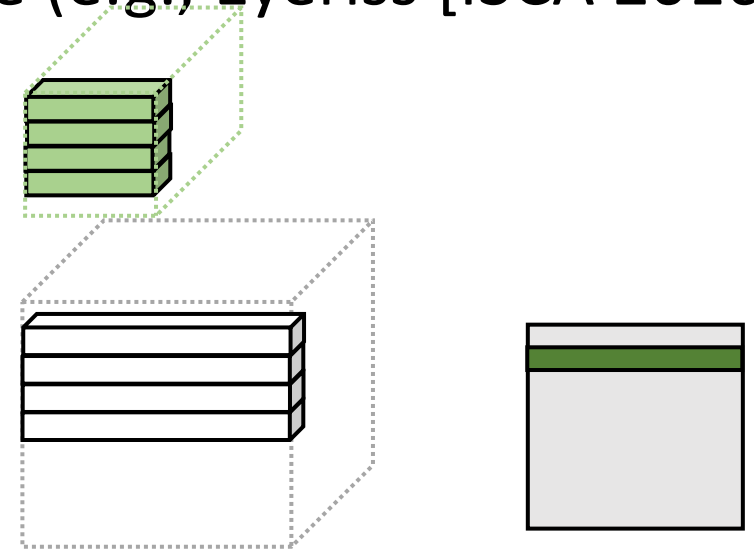
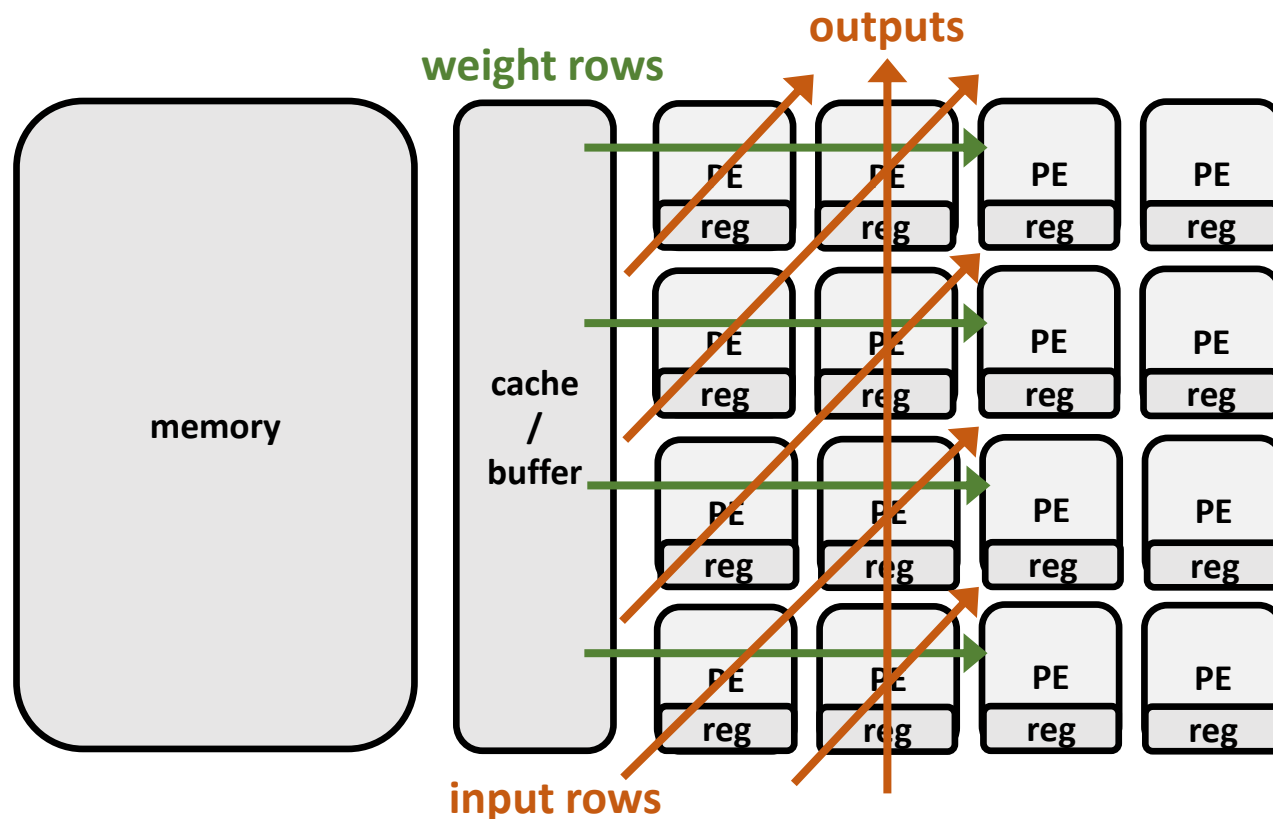
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

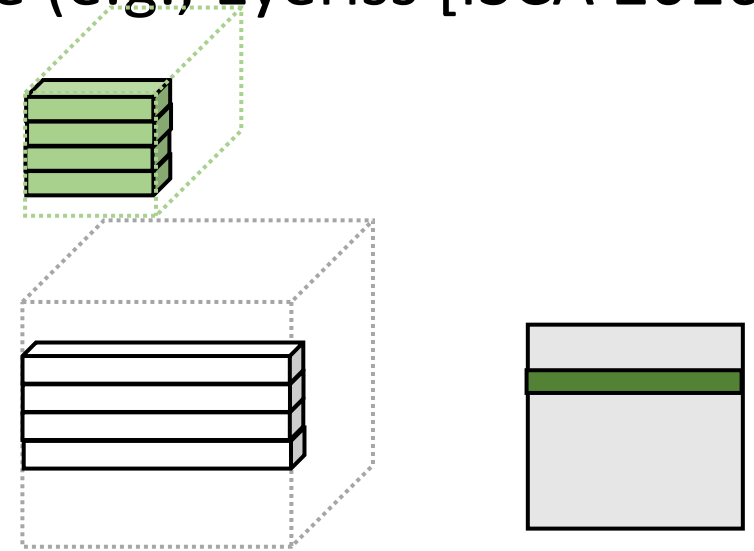
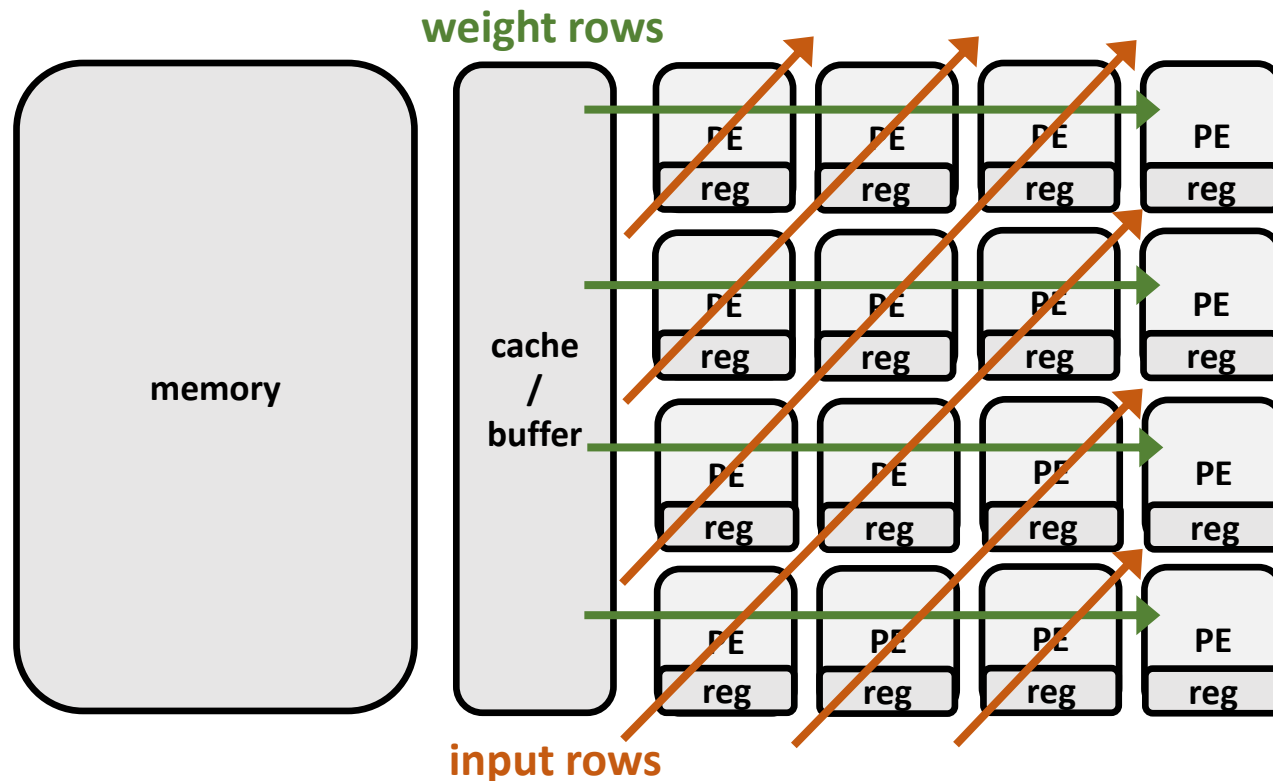
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

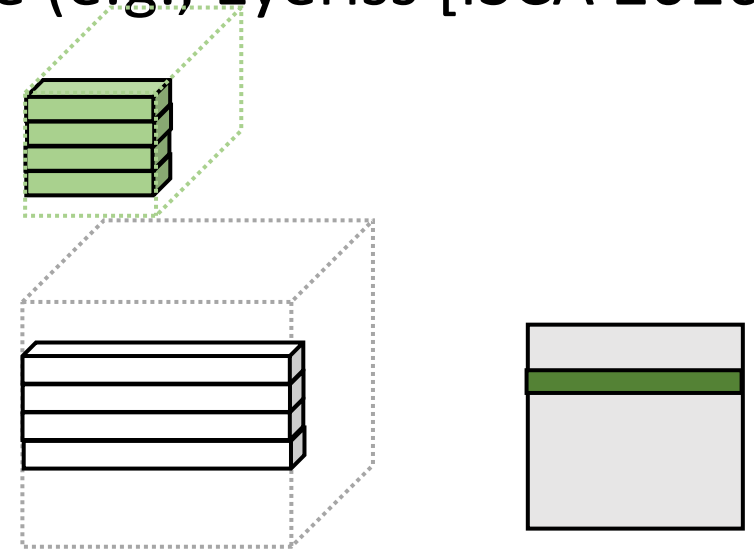
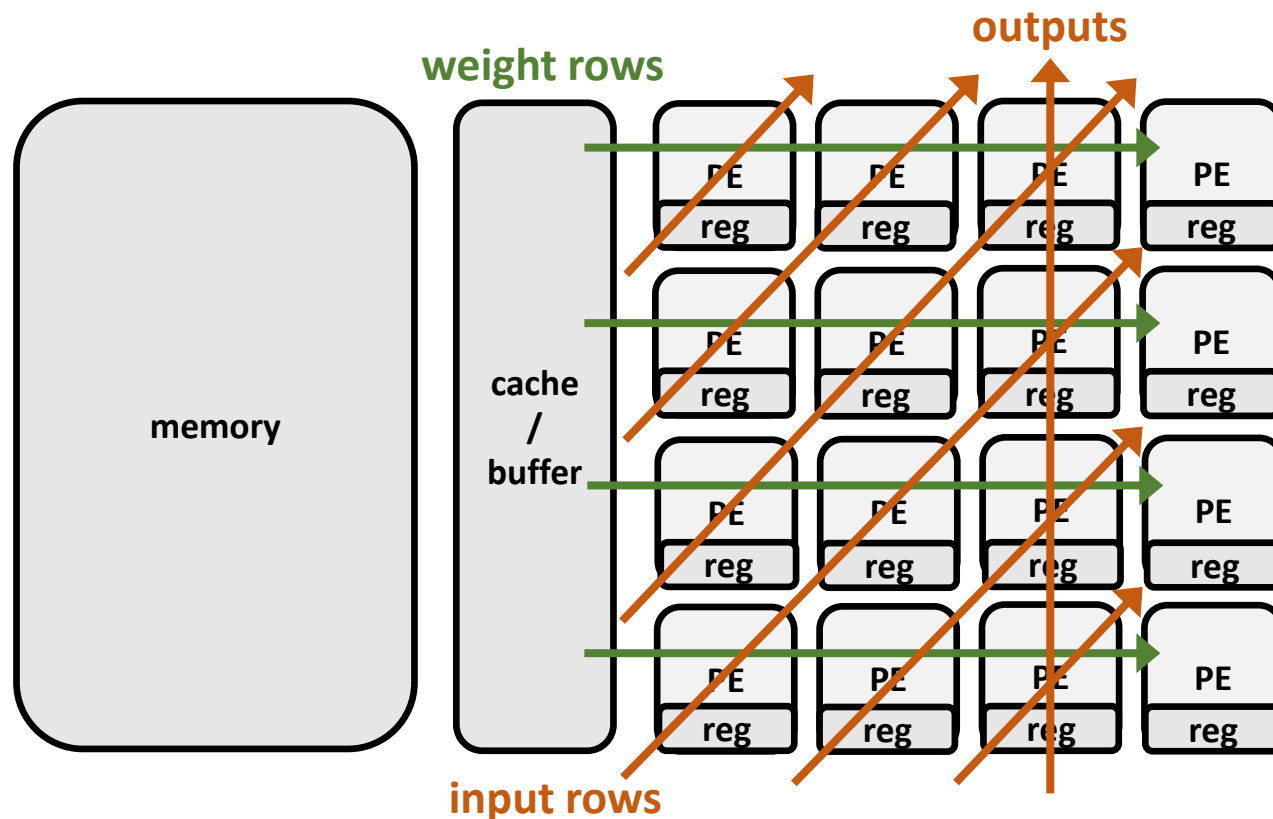
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

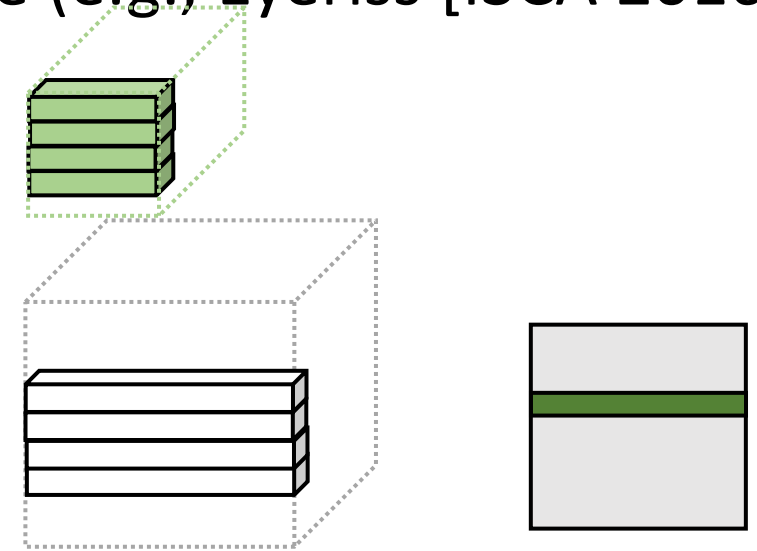
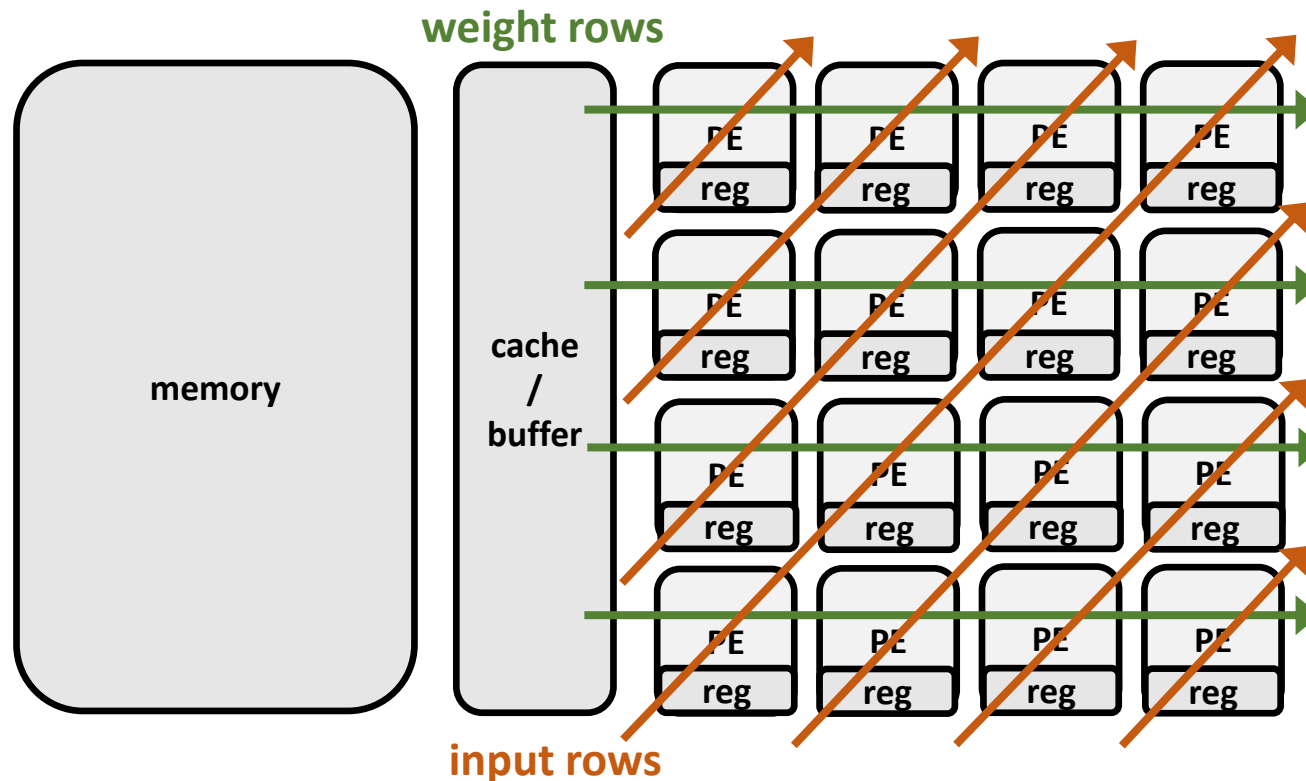
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

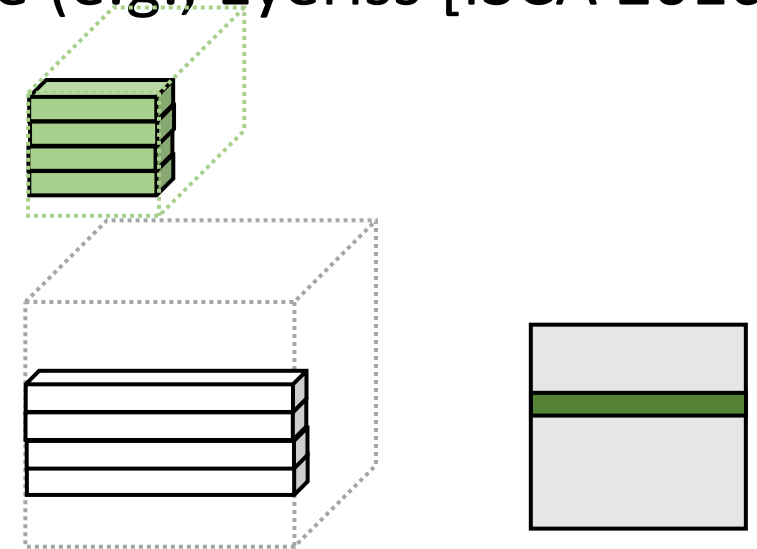
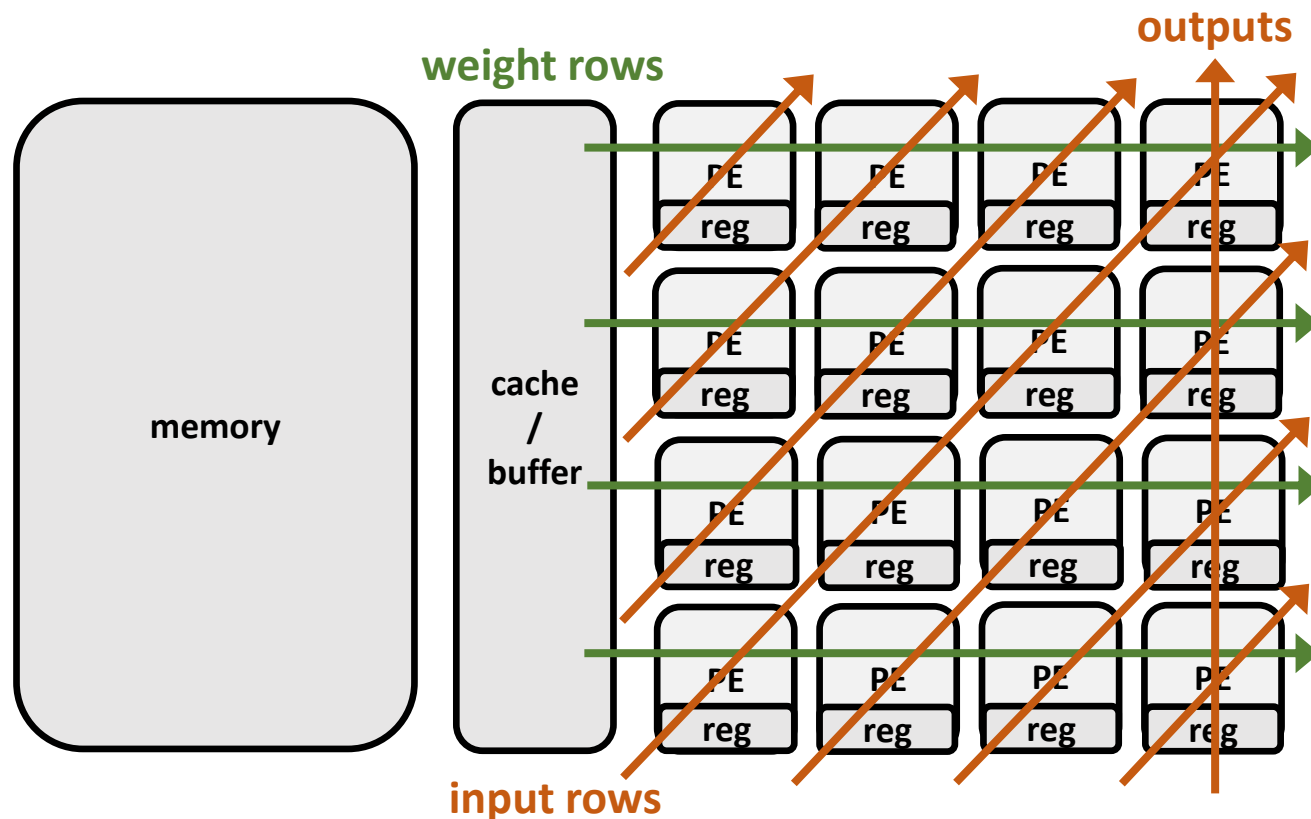
- Effective if layer configurations vary significantly



Neural Network Accelerators – Architecture

Row-Stationary Dataflow: Strikes a balance of reuse (e.g., Eyeriss [ISCA 2016])

- Effective if layer configurations vary significantly



Carat

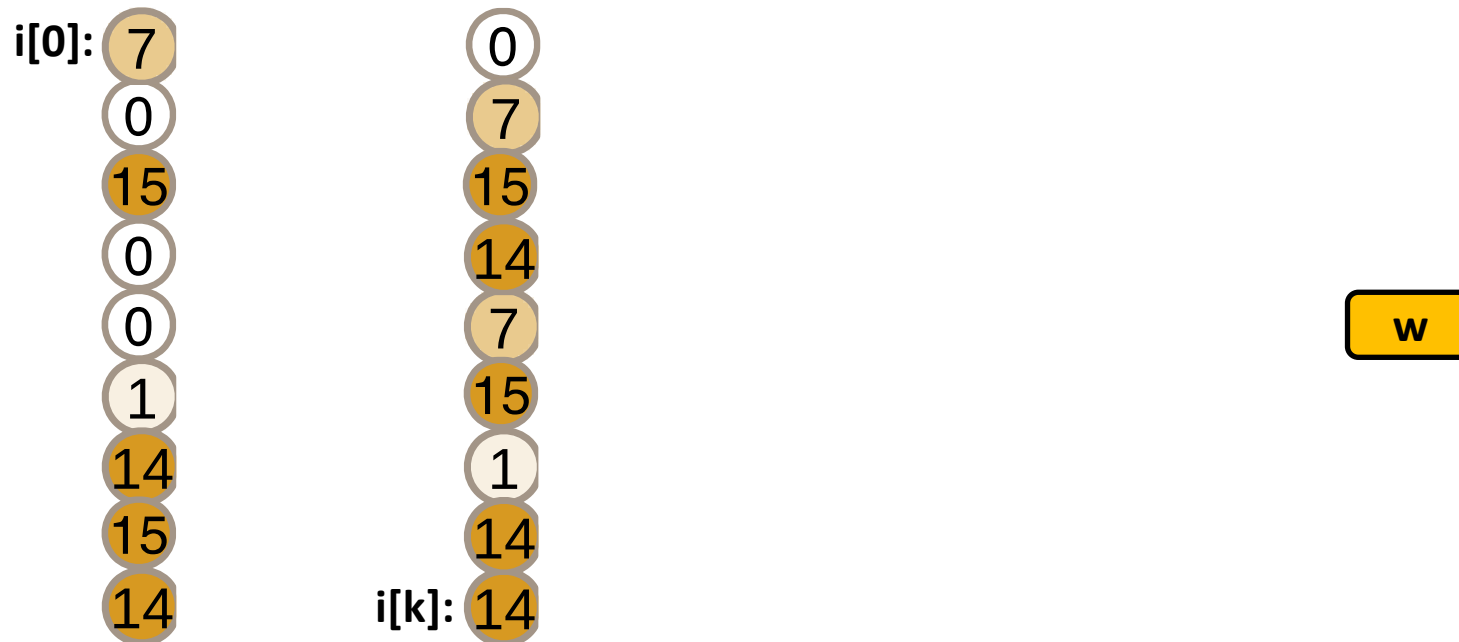
Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values

Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

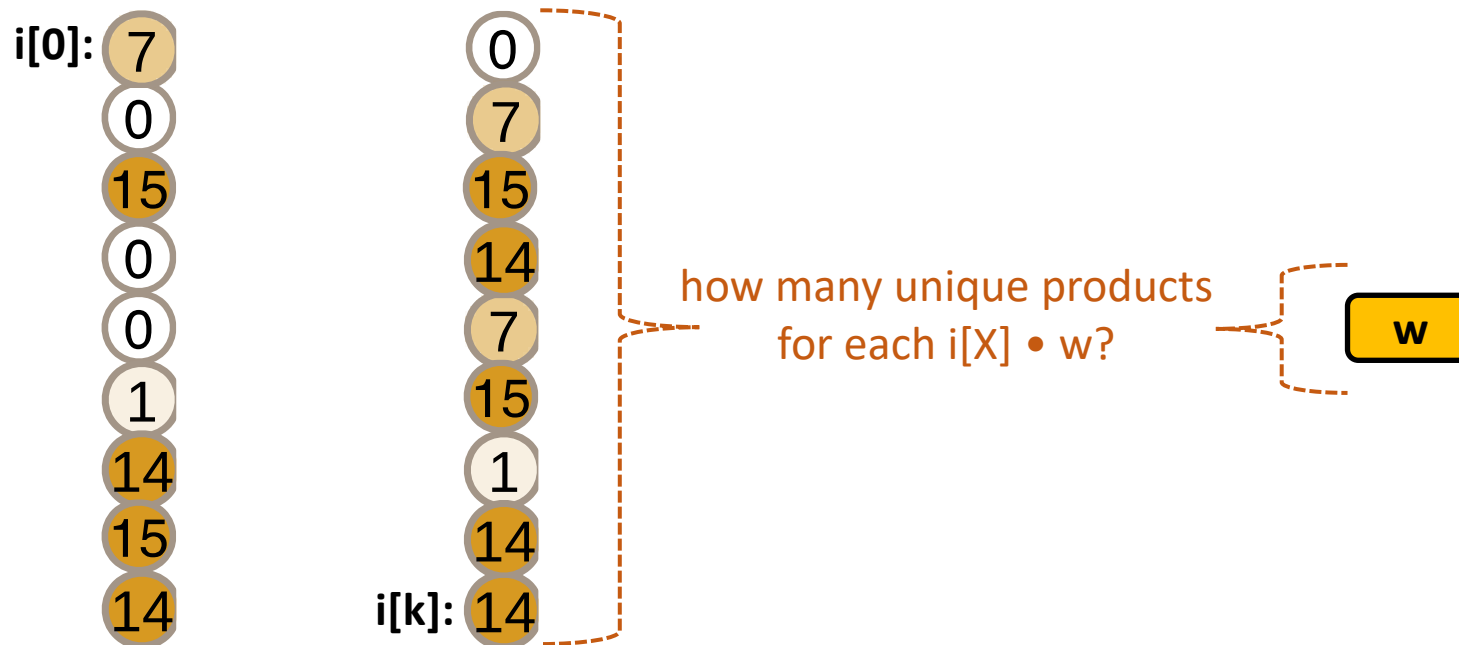
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

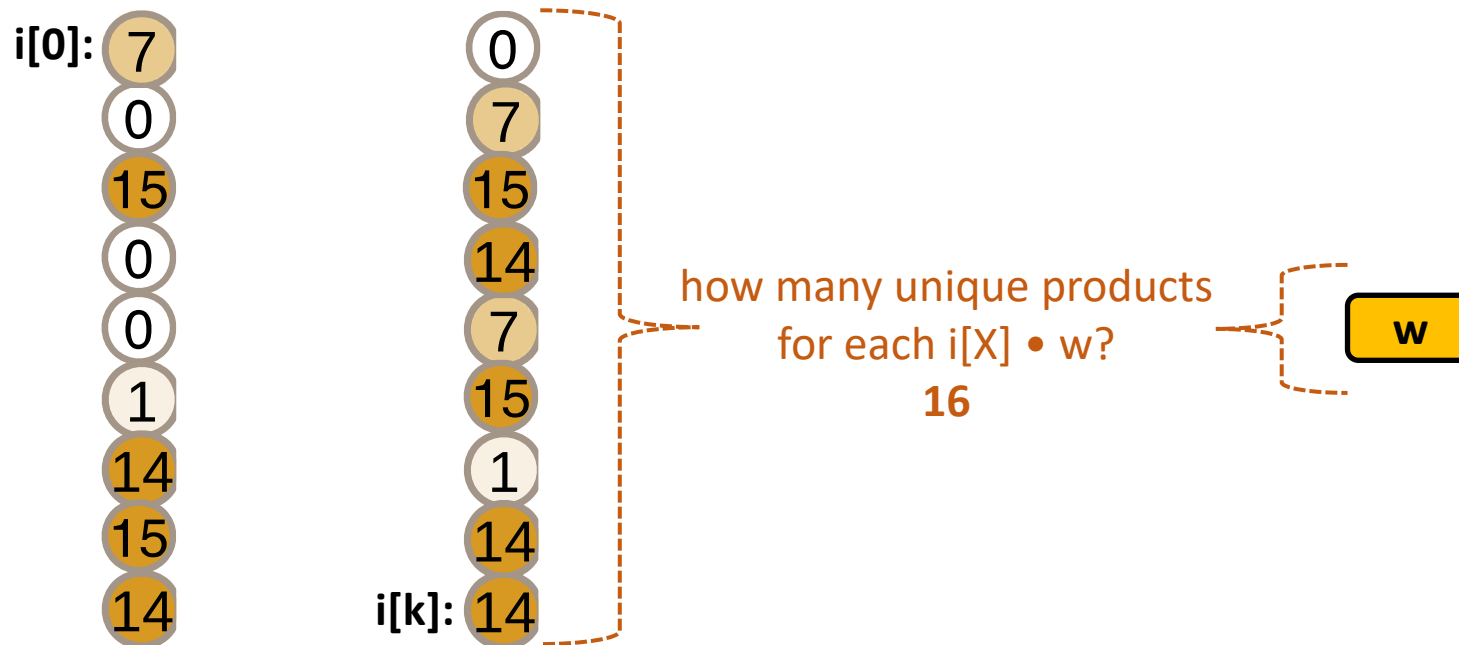
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

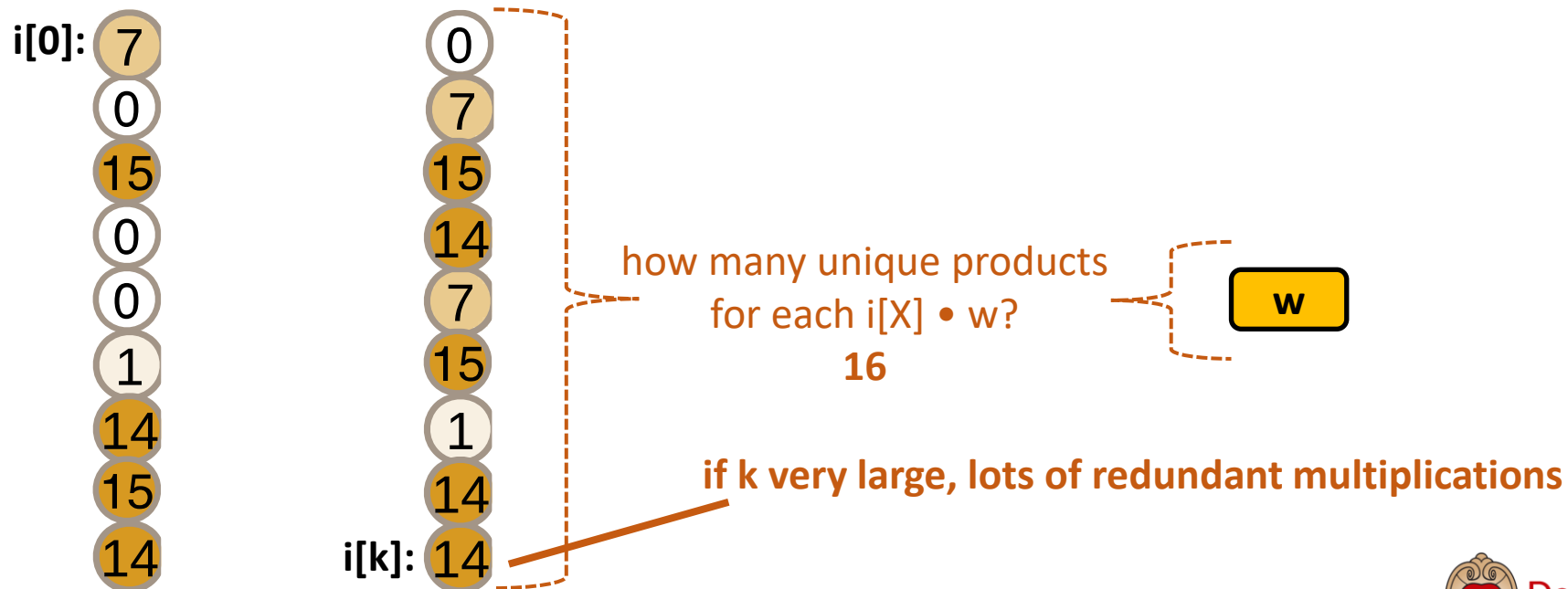
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

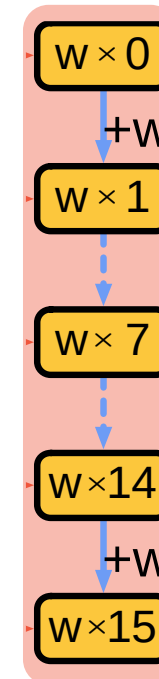
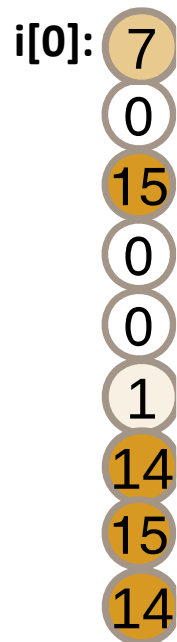
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

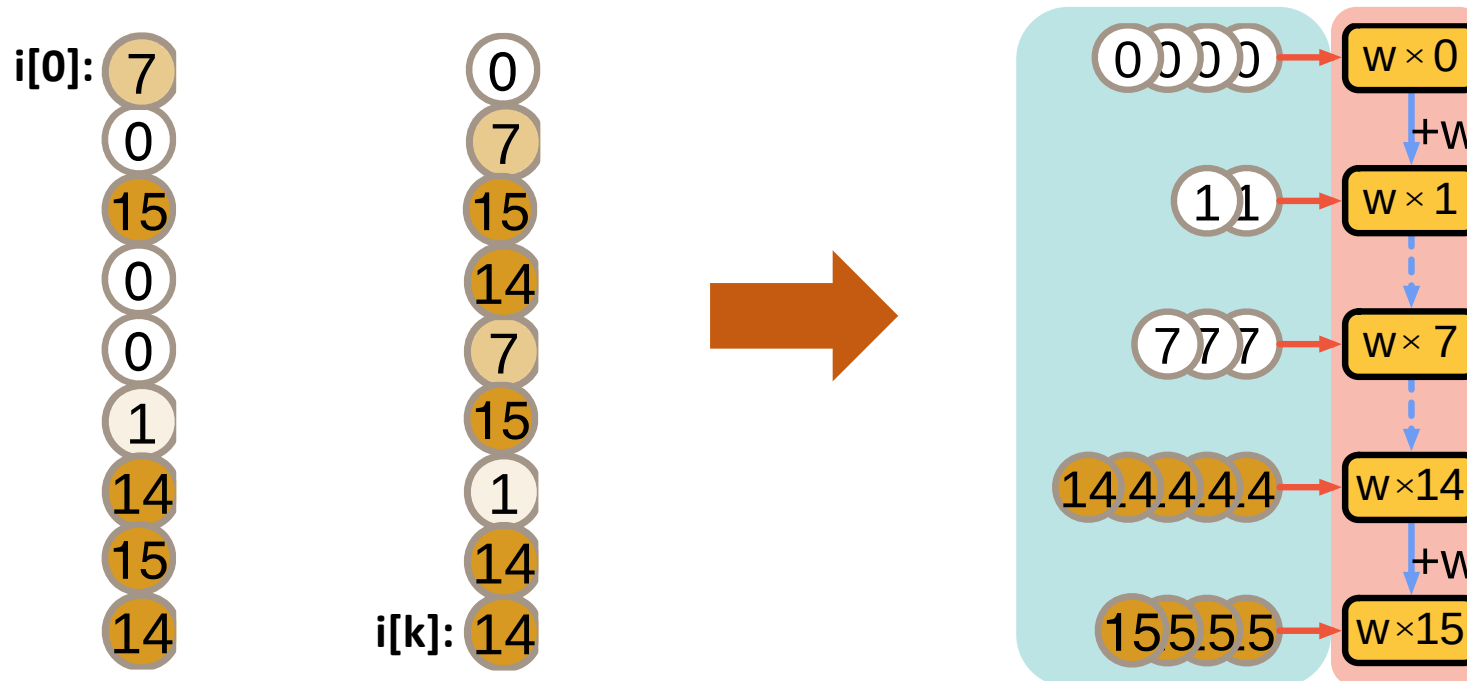
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

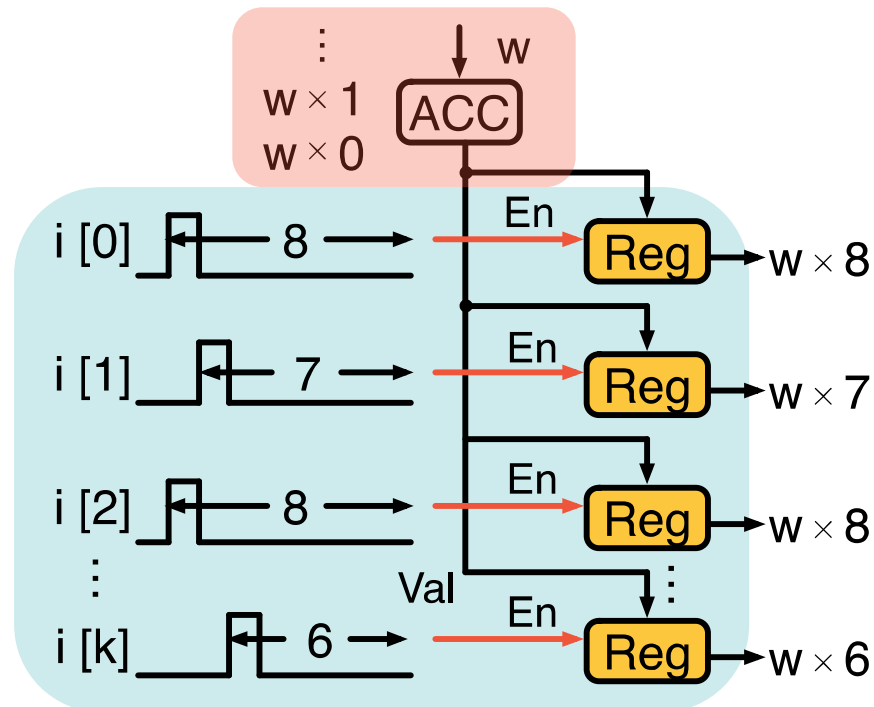
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

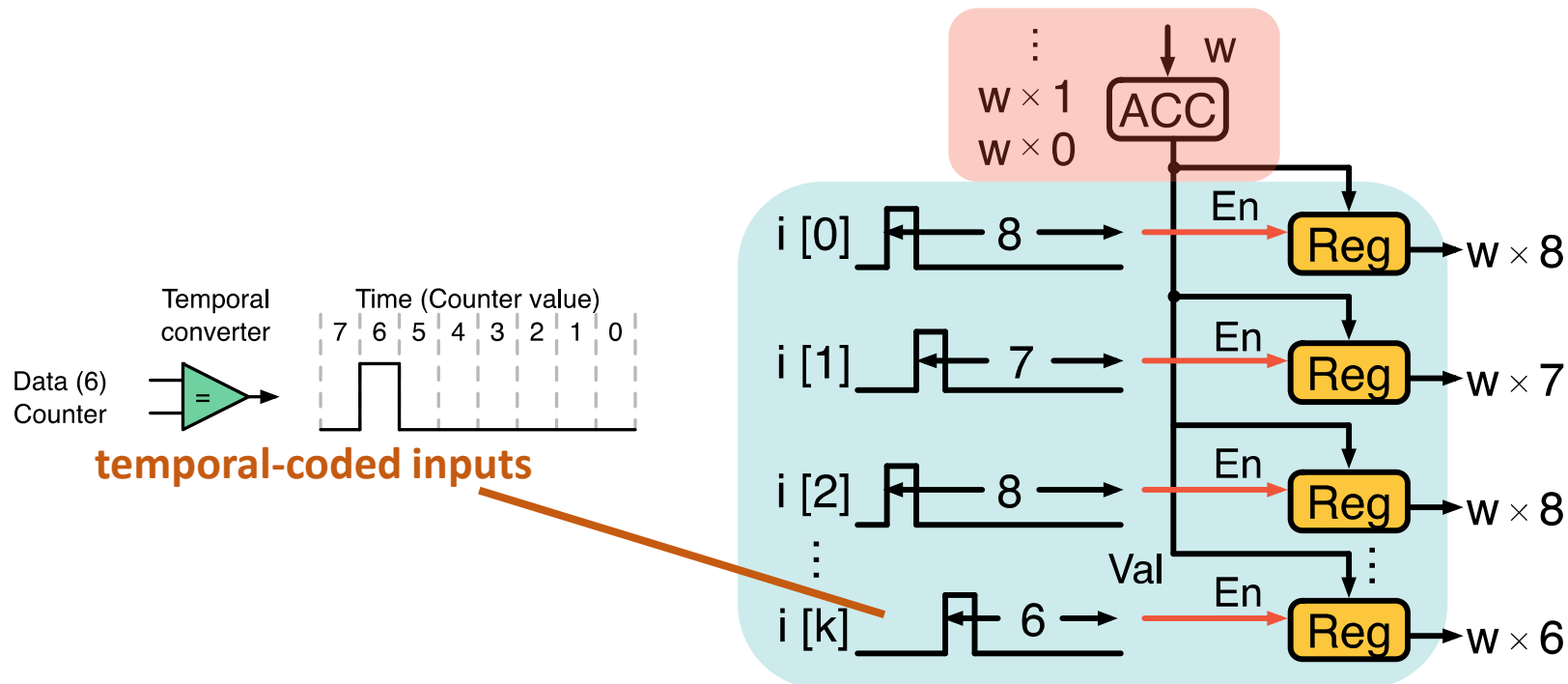
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

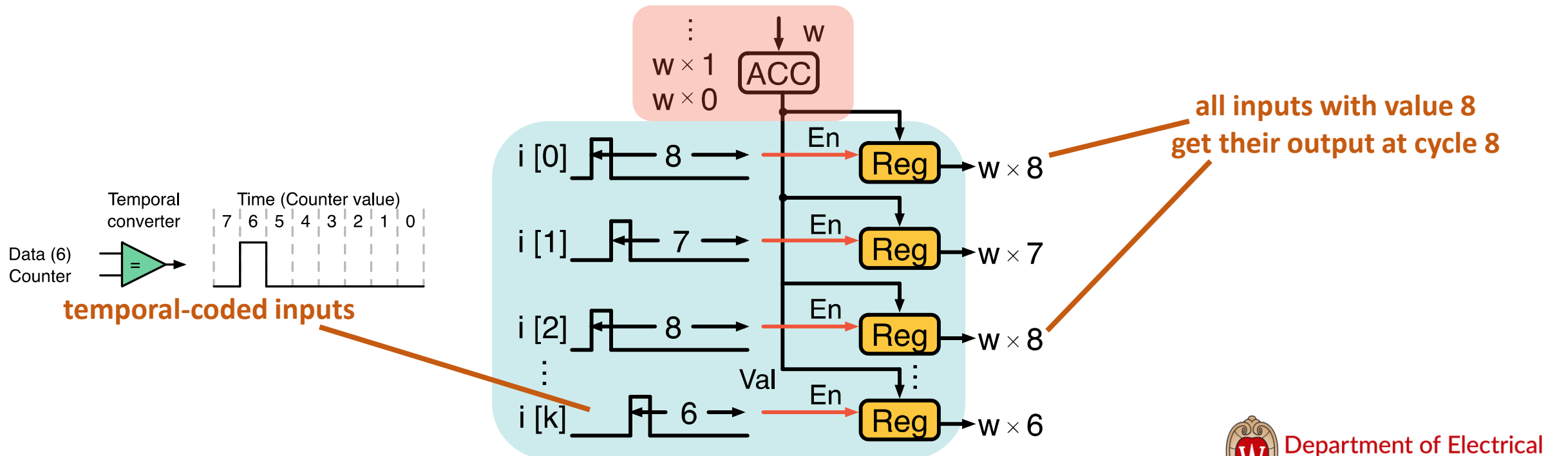
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

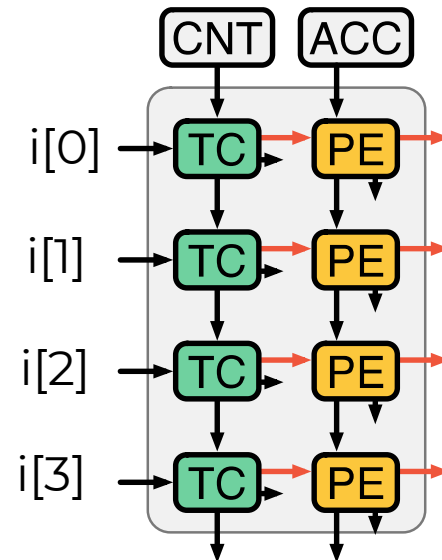
- Leverages trends towards lower precision (e.g., fp8, int4)
- Introduces a form of *value-level parallelism*: inputs *subscribe* to output values
- e.g., Vector-scalar multiplication: $i[] \bullet w$ (4-bit)



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

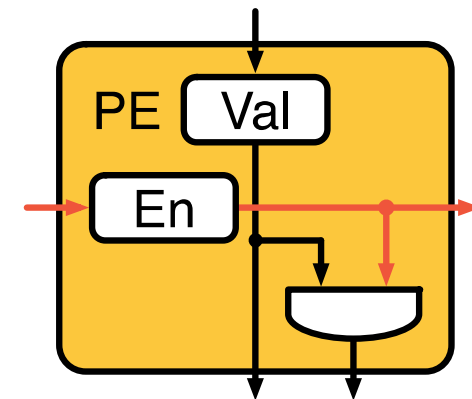
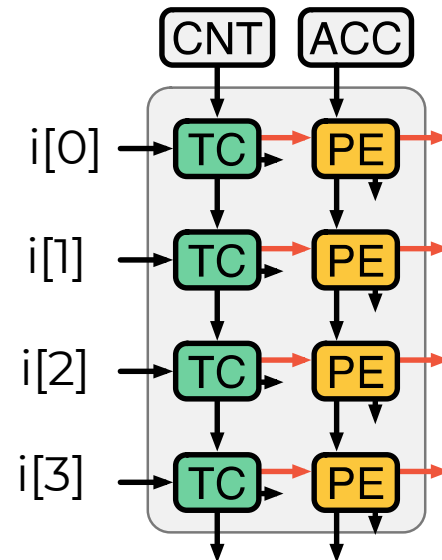
- Multiplicands are accumulated and pipelined down a column of PEs



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

- Multiplicands are accumulated and pipelined down a column of PEs



Carat

Accelerator for NN computations without multiplier hardware [ASPLOS 2024]:

- Multiplicands are accumulated and pipelined down a column of PEs
- Multipliers are temporal-coded and staggered (i.e., $i[x+1]$ starts one cycle after $i[x]$)

